colorbrewer.js and colorbrewer.css for this purpose. Each array in colorbrewer.js corresponds to one of Brewer's color schemes, designed for a set number of colors. For instance, the reds scale looks like this:

```
Reds: {
3: ["#fee0d2","#fc9272","#de2d26"],
4: ["#fee5d9","#fcae91","#fb6a4a","#cb181d"],
5: ["#fee5d9","#fcae91","#fb6a4a","#de2d26","#a50f15"],
6: ["#fee5d9","#fcbba1","#fc9272","#fb6a4a","#de2d26","#a50f15"],
7: ["#fee5d9","#fcbba1","#fc9272","#fb6a4a","#ef3b2c","#cb181d","#99000d"],
8: ["#fff5f0","#fee0d2","#fcbba1","#fc9272",
    "#fb6a4a","#ef3b2c","#cb181d","#99000d"],
9: ["#fff5f0","#fee0d2","#fcbba1","#fc9272","#fb6a4a",
    "#ef3b2c","#cb181d","#a50f15","#67000d"]
}
```

This provides high-legibility, discrete colors in the red spectrum for our elements. Again, we'll color your circles by region, but this time, we'll color them by their magnitude using our `buttonClick` function. We need to use the quantize scale that you saw earlier in chapter 2, because the colorbrewer scales, despite being discrete scales, are designed for quantitative data that has been separated into categories. In other words, they're built for numerical data, but numerical data that has been sorted into ranges, such as when you break down all the ages of adults in a census into categories of 18–35, 36–50, 51–65, and 65+.

```
function buttonClick(datapoint) {
    var maxValue = d3.max(incomingData, function(el) {
            return parseFloat(el[datapoint]);
        });
    var colorQuantize = d3.scale.quantize()
            .domain([0,maxValue]).range(colorbrewer.Reds[3]);
    var radiusScale = d3.scale.linear()
            .domain([0,maxValue]).range([2,20]);
    d3.selectAll("g.overallG").select("circle").transition().duration(1000)
        .style("fill", function(p) {
            return colorQuantize(p[datapoint]);
        }).attr("r", function(p) {
            return radiusScale(p[datapoint]);
        });
};
```

Our new buttonClick function sorts the circles in our visualization into three categories with colors associated with them.

The quantize scale sorts the numerical data into as many categories as there are in the range. Because colorbrewer.Reds[3] is an array of three values, the dataset is sorted into three discrete categories, and each category has a different shade of red assigned.

One of the conveniences of using colorbrewer.js dynamically paired to a quantizing scale is that if we adjust the number of colors, for instance, from `colorbrewer.Reds[3]` (shown in figure 3.17) to `colorbrewer.Reds[5]`, the range of numerical data is represented with five colors instead of three.